Vector Search

Vector Search

With the rise of Generative AI, Vector databases have gained strong traction in the world of databases. These databases enable efficient storage and querying of high-dimensional vectors, making them well-suited for tasks such as semantic search, recommendation systems, and natural language understanding.

Vector search is a technique that retrieves semantically similar data by comparing vector representations (also known as embeddings) rather than relying on traditional exact-match queries. This approach enables intelligent, context-aware applications that go beyond keyword-based retrieval.

In the context of Spring Data, vector search opens new possibilities for building intelligent, context-aware applications, particularly in domains like natural language processing, recommendation systems, and generative AI. By modelling vector-based querying using familiar repository abstractions, Spring Data allows developers to seamlessly integrate similarity-based vector-capable databases with the simplicity and consistency of the Spring Data programming model.

To use Hibernate Vector Search, you need to add the following dependencies to your project.

The following example shows how to set up dependencies in Maven and Gradle:

Maven | **Gradle**

XML

```xml
<dependencies>
    <dependency>
      <groupId>org.hibernate.orm</groupId>
      <artifactId>hibernate-vector</artifactId>
```

```
      <version>${hibernate.version}</version>
    </dependency>
</dependencies>
```

NOTE

While you can use `Vector` as type for queries, you cannot use it in your domain model as Hibernate requires float or double arrays as vector types.

Vector Model

To support vector search in a type-safe and idiomatic way, Spring Data introduces the following core abstractions:

- `Vector`
- `SearchResults<T>` and `SearchResult<T>`
- `Score`, `Similarity` and Scoring Functions

`Vector`

The `Vector` type represents an n-dimensional numerical embedding, typically produced by embedding models. In Spring Data, it is defined as a lightweight wrapper around an array of floating-point numbers, ensuring immutability and consistency. This type can be used as an input for search queries or as a property on a domain entity to store the associated vector representation.

JAVA

```
Vector vector = Vector.of(0.23f, 0.11f, 0.77f);
```

Using `Vector` in your domain model removes the need to work with raw arrays or lists of numbers, providing a more type-safe and expressive way to handle vector data. This abstraction also allows for easy integration with various vector databases and libraries. It also allows for implementing vendor-specific optimizations such as binary or quantized vectors that do not map to a standard floating point (`float` and `double` as of IEEE 754) representation. A domain object can have a vector property, which can be used for similarity searches. Consider the following example:

JAVA

```
class Comment {

  @Id String id;
  String country;
  String comment;
```

```
  @Column(name = "the_embedding")
  @JdbcTypeCode(SqlTypes.VECTOR)
  @Array(length = 5)
  Vector embedding;

  // getters, setters, …
}
```

NOTE

Associating a vector with a domain object results in the vector being loaded and stored as part of the entity life-cycle, which may introduce additional overhead on retrieval and persistence operations.

Search Results

The `SearchResult<T>` type encapsulates the results of a vector similarity query. It includes both the matched domain object and a relevance score that indicates how closely it matches the query vector. This abstraction provides a structured way to handle result ranking and enables developers to easily work with both the data and its contextual relevance.

Example 1. Using `SearchResult<T>` in a Repository Search Method

JAVA

```
interface CommentRepository extends Repository<Comment, String> {

  SearchResults<Comment> searchByCountryAndEmbeddingNear(String country, Vector vector,
Score distance,
    Limit limit);

  @Query("""
      SELECT c, cosine_distance(c.embedding, :embedding) as distance FROM Comment c
      WHERE c.country = ?1
        AND cosine_distance(c.embedding, :embedding) <= :distance
      ORDER BY distance asc""")
  SearchResults<Comment> searchAnnotatedByCountryAndEmbeddingWithin(String country,
Vector embedding,
      Score distance);
}

SearchResults<Comment> results = repository.searchByCountryAndEmbeddingNear("en",
Vector.of(…), Score.of(0.9), Limit.of(10));
```

In this example, the `searchByCountryAndEmbeddingNear` method returns a `SearchResults<Comment>` object, which contains a list of `SearchResult<Comment>` instances. Each result includes the matched

`Comment` entity and its relevance score.

Relevance score is a numerical value that indicates how closely the matched vector aligns with the query vector. Depending on whether a score represents distance or similarity a higher score can mean a closer match or a more distant one.

The scoring function used to calculate this score can vary based on the underlying database, index or input parameters.

Score, Similarity, and Scoring Functions

The `Score` type holds a numerical value indicating the relevance of a search result. It can be used to rank results based on their similarity to the query vector. The `Score` type is typically a floating-point number, and its interpretation (higher is better or lower is better) depends on the specific similarity function used. Scores are a by-product of vector search and are not required for a successful search operation. Score values are not part of a domain model and therefore represented best as out-of-band data.

Generally, a Score is computed by a `ScoringFunction`. The actual scoring function used to calculate this score can depends on the underlying database and can be obtained from a search index or input parameters.

Spring Data support declares constants for commonly used functions such as:

Euclidean Distance

Calculates the straight-line distance in n-dimensional space involving the square root of the sum of squared differences.

Cosine Similarity

Measures the angle between two vectors by calculating the Dot product first and then normalizing its result by dividing by the product of their lengths.

Dot Product

Computes the sum of element-wise multiplications.

The choice of similarity function can impact both the performance and semantics of the search and is often determined by the underlying database or index being used. Spring Data adopts to the database's native scoring function capabilities and whether the score can be used to limit results.

Hibernate translates distance function calls to native database functions for PGvector and Oracle. Their result is typically a distance. When using `Similarity` instead of `Score`, Spring Data normalizes dis-

tance scores into a similarity score between 0 and 1. The higher the score, the more similar the two vectors are.

Example 2. Using `Score` and `Similarity` in a Repository Search Methods

```java
interface CommentRepository extends Repository<Comment, String> {

  SearchResults<Comment> searchByEmbeddingNear(Vector vector, ScoringFunction function);

  SearchResults<Comment> searchByEmbeddingNear(Vector vector, Score score);

  SearchResults<Comment> searchByEmbeddingNear(Vector vector, Similarity similarity);

  SearchResults<Comment> searchByEmbeddingNear(Vector vector, Range<Similarity> range);
}

repository.searchByEmbeddingNear(Vector.of(…), ScoringFunction.cosine()); 1

repository.searchByEmbeddingNear(Vector.of(…), Score.of(0.9, ScoringFunction.cosine())); 2

repository.searchByEmbeddingNear(Vector.of(…), Similarity.of(0.9, ScoringFunction.cosine())); 3

repository.searchByEmbeddingNear(Vector.of(…), Similarity.between(0.5, 1, ScoringFunction.euclidean())); 4
```

1. Run a search and return results that are similar to the given `Vector` applying Cosine scoring.
2. Run a search and return results with a score of `0.9` or smaller using the Cosine distance.
3. Run a search and normalize the score into a similarity value. Return results with a similarity of `0.9` or greater using Cosine scoring.
4. Run a search and normalize the score into a similarity value. Return results with a similarity of between `0.5` and `1.0` or greater using Euclidean scoring.

NOTE

JPA requires a `ScoringFunction` to be provided when creating `Score` or `Similarity` instances to select a scoring function.

Vector Search Methods

Vector search methods are defined in repositories using the same conventions as standard Spring Data query methods. These methods return `SearchResults<T>` and require a `Vector` parameter to define the query vector. The actual implementation depends on the actual internals of the underlying data store and its capabilities around vector search.

NOTE

If you are new to Spring Data repositories, make sure to familiarize yourself with the basics of repository definitions and query methods.

Generally, you have the choice of declaring a search method using two approaches:

- Query Derivation
- Declaring a String-based Query

Vector Search methods must declare a `Vector` parameter to define the query vector.

Derived Search Methods

A derived search method uses the name of the method to derive the query. Vector Search supports the following keywords to run a Vector search when declaring a search method:

Table 1. Query predicate keywords

Logical keyword	Keyword expressions
`NEAR`	`Near`, `IsNear`
`WITHIN`	`Within`, `IsWithin`

Example 3. Using `Near` and `Within` Keywords in Repository Search Methods

JAVA

```
interface CommentRepository extends Repository<Comment, String> {

  SearchResults<Comment> searchByEmbeddingNear(Vector vector, Score score);

  SearchResults<Comment> searchByEmbeddingWithin(Vector vector, Range<Similarity>
range);

  SearchResults<Comment> searchByCountryAndEmbeddingWithin(String country, Vector
vector, Range<Similarity> range);
}
```

Derived search methods can declare predicates on domain model attributes and Vector parameters.

Derived search methods are typically easier to read and maintain, as they rely on the method name to express the query intent. However, a derived search method requires either to declare a `Score`, `Range<Score>` or `ScoreFunction` as second argument to the `Near` / `Within` keyword to limit search results by their score.

Annotated Search Methods

Annotated methods provide full control over the query semantics and parameters. Unlike derived methods, they do not rely on method name conventions.

Annotated search methods must define the entire JPQL query to run a Vector Search.

Example 4. Using @Query Search Methods

```java
interface CommentRepository extends Repository<Comment, String> {

  @Query("""
      SELECT c, cosine_distance(c.embedding, :embedding) as distance FROM Comment c
      WHERE c.country = ?1
        AND cosine_distance(c.embedding, :embedding) <= :distance
      ORDER BY distance asc""")
  SearchResults<Comment> searchAnnotatedByCountryAndEmbeddingWithin(String country,
Vector embedding,
      Score distance);

  @Query("""
      SELECT c FROM Comment c
      WHERE c.country = ?1
        AND cosine_distance(c.embedding, :embedding) <= :distance
      ORDER BY cosine_distance(c.embedding, :embedding) asc""")
  List<Comment> findAnnotatedByCountryAndEmbeddingWithin(String country, Vector
embedding, Score distance);
}
```

Vector Search methods are not required to include a score or distance in their projection. When using annotated search methods returning `SearchResults`, the execution mechanism assumes that if a second projection column is present that this one holds the score value.

With more control over the actual query, Spring Data can make fewer assumptions about the query and its parameters. For example, `Similarity` normalization uses the native score function within the query to normalize the given similarity into a score predicate value and vice versa. If an annotated query does not define e.g. the score, then the score value in the returned `SearchResult<T>` will be zero.

Sorting

By default, search results are ordered according to their score. You can override sorting by using the `Sort` parameter:

Example 5. Using `Sort` in Repository Search Methods

```java
interface CommentRepository extends Repository<Comment, String> {

  SearchResults<Comment> searchByEmbeddingNearOrderByCountry(Vector vector, Score score);

  SearchResults<Comment> searchByEmbeddingWithin(Vector vector, Score score, Sort sort);
}
```

Please note that custom sorting does not allow expressing the score as a sorting criteria. You can only refer to domain properties.